Z-Image World Model: Action-Conditioned Video Generation for Game Simulation via Frozen Diffusion Transformers

inZOI Simulation Team
ugonfor@krafton.com

March 2026

Abstract

We present Z-Image World Model, an action-conditioned video generation system for game simulation built on top of the frozen Z-Image-Turbo diffusion transformer (6.15B parameters). Our approach wraps the pretrained model with lightweight trainable modules—TemporalAttention layers (one per transformer block, 30 total) and ActionInjection cross-attention layers (at depths 1/4, 1/2, 3/4)—preserving the base model's visual quality while enabling temporal coherence and player action conditioning. Starting from a frozen pretrained image generator, we train in three stages: (1) temporal layer warmup on diverse video data, (2) action identity learning on GameFactory Minecraft data using a contrastive loss, and (3) continued fine-tuning with fixed bugs in initialization and loss formulation. We identify and fix three critical training bugs—a zero-initialization deadlock in cross-attention, a zero-gradient contrastive loss, and bfloat16 precision overflow in temporal gating—that together caused a 7× improvement in action responsiveness (ratio: 0.065 → 0.463). Further training achieves a final responsiveness ratio of 0.5725, confirming that action conditioning generates measurably different outputs for different player inputs. We describe the training methodology, architectural choices, failure modes encountered, and an unexpected K/V amplification mechanism by which the model learned action discrimination without to_out weight growth.

1 Introduction

Building a world model—a generative model that simulates the future state of an environment given player actions—is a core challenge in game AI. Unlike offline video generation, a game world model must:

1. Generate temporally coherent video (no flickering, smooth motion)
2. Respond distinctly to different player actions (forward ≠ backward)
3. Run at interactive frame rates
4. Maintain the visual quality of the base game

Modern large-scale text-to-image diffusion models encode rich visual priors. We hypothesize that these priors can be extended to video generation and action conditioning by attaching small trainable modules, while keeping the 6.15B pretrained weights frozen. This avoids catastrophic forgetting and dramatically reduces the number of parameters to train (∼784M vs. 8.2B total).

Our base model is Z-Image-Turbo (Tongyi-MAI), a single-stream S3-DiT architecture with 30 transformer layers, hidden dimension 3840, and RoPE positional encoding. We extend it with:

- TemporalAttention: causal cross-frame attention after each transformer block
- ActionInjectionLayer: action-conditioned cross-attention residual at three depths
- ActionEncoder: learned 512→3840-dim action embedding projection

Training proceeds in stages using FIFO-Diffusion [3] for streaming inference, GameFactory [5] Minecraft data for action-labeled clips, and a contrastive action consistency loss to enforce action identity.

2 Related Work

Video World Models. Genie [1] and GameNGen [4] demonstrate that diffusion models can simulate game environments frame-by-frame, but train from scratch on game-specific data. Our approach instead adapts a large pretrained image model, reducing the data requirement.

Action-Conditioned Video Generation. UniSim [6] and IRASim [8] condition video generation on robot/human actions via cross-attention or AdaLN. We adopt cross-attention injection (ActionInjectionLayer) at multiple depths, matching the architecture of ControlNet [7] applied to temporal video.

Efficient Fine-tuning of Diffusion Models. LoRA [2] and ControlNet [7] show that small adapter modules can effectively steer large frozen models. Our TemporalAttention and ActionInjection modules follow this paradigm: zero-initialized so that at the start of training, the model behaves identically to the frozen base.

FIFO-Diffusion. FIFO-Diffusion [3] enables streaming video generation using a queue of latents at varying noise levels, allowing infinite-length generation without recomputing the full history. We adopt this for inference.

3 Architecture

3.1 Base Model: Z-Image-Turbo

Z-Image-Turbo is a 6.15B-parameter S3-DiT (Single-Stream Diffusion Transformer). Its architecture:

- 30 main transformer layers + 2 noise refiners + 2 context refiners
- Hidden dim $D = 3840$, 30 heads, head dim 128
- RMSNorm (sandwich), QK-Norm, RoPE positional encoding
- SwiGLU FFN: $3840 \to 10240 \to 3840$
- 4-way adaLN modulation ($256 \to 15360$)
- Single-stream: image and text tokens concatenated

All 6.15B parameters are frozen throughout training.

3.2 TemporalAttention

After each of the 30 transformer layers, we insert a TemporalAttention module:

$$\mathbf{h}'_t = \mathbf{h}_t + \gamma \cdot \text{CausalAttn}(\text{RMSNorm}(\mathbf{h}_{1:t})) \quad (1)$$

where γ is a per-layer learnable scalar initialized to 0. The zero initialization ensures the model starts as the frozen base; γ gradually grows as the temporal module learns useful structure.

The causal mask allows each frame to attend to all previous frames, enabling temporal coherence without future information leakage.

bfloat16 Precision Issue. Early training used $\text{lr}_{\text{temporal}} = 5 \times 10^{-6}$, but gammas (initialized near 0) could not update: bfloat16's minimum representable step at magnitudes $\sim 10^{-2}$ is $\sim 1.2 \times 10^{-4}$, larger than the learning rate. Fix: cast gamma parameters to float32 at training start and use $\text{lr}_{\text{temporal}} \geq 10^{-4}$.

3.3 ActionInjectionLayer

Action conditioning is applied at transformer layers 7, 15, and 22 (depths 1/4, 1/2, 3/4):

$$\mathbf{r} = \text{CrossAttn}(\mathbf{x}, \mathbf{a}) \cdot W_{\text{out}} \quad (2)$$

$$\mathbf{x}' = \mathbf{x} + \sigma(\text{gate}) \cdot \mathbf{r} \quad (3)$$

where $\mathbf{a} = \text{ActionEncoder}(\text{action})$ is the 3840-dim action embedding and gate is a learnable scalar.

Zero-init Deadlock. Early versions zero-initialized W_{out}. With $W_{\text{out}} = 0$: the residual $\mathbf{r} = 0$, the gate gradient $\partial L/\partial \text{gate} = \sigma'(\text{gate}) \cdot \mathbf{r} = 0$. Both W_{out} and gate are locked at zero; neither can receive gradients through the other. Fix: $W_{\text{out}} \sim \mathcal{U}(-a, a)$ with $a = \text{gain}/\sqrt{\text{fan_in}}$, gain $= 0.01$ (Xavier uniform).

3.4 ActionEncoder

The ActionEncoder maps a discrete action index to a 3840-dim conditioning vector:

$$\mathbf{a} = \text{LayerNorm}(\text{Linear}_{512\to3840}(\text{Embed}_{17\times512}(\text{action}))) \quad (4)$$

We define 17 discrete actions corresponding to combinations of WASD movement, sprint, jump, and interactions (Table 1).

Table 1: Discrete action space (17 actions).

Index	Action
0	IDLE
1	MOVE_FORWARD
2	MOVE_BACKWARD
3	MOVE_LEFT
4	MOVE_RIGHT
5	RUN_FORWARD
6	JUMP
7–10	Diagonal moves (F+L, F+R, B+L, B+R)
11–16	Diagonal run variants

Table 2: Model parameter summary.

Component	Params	Trainable
Z-Image-Turbo (frozen)	6.15B	No
TemporalAttention (30 layers)	686M	Yes
ActionInjection (3 layers)	89M	Yes
ActionEncoder	9.4M	Yes
Total	8.2B	784M

3.5 Parameter Counts

4 Training

4.1 Stage 1: Temporal Layer Warmup

Data: Diverse internet video clips (various sources).
Loss: Diffusion forcing loss (MSE on predicted velocity $\mathbf{v}$).
Goal: TemporalAttention layers learn to produce temporally coherent frames from the frozen spatial features.
Result: Loss 0.748 $\rightarrow$ 0.676 (best) over 10 epochs. Gammas grow from 0 to $\sim$0.03, indicating temporal structure is being learned.

4.2 Stage 2: Action Identity Learning

Data: GameFactory Minecraft dataset [5] (274 clips, action-labeled).
Loss:

$$\mathcal{L} = \mathcal{L}_{\text{diffusion}} + \lambda_c \mathcal{L}_{\text{contrastive}} \tag{5}$$

Contrastive loss:

$$\mathcal{L}_{\text{contrastive}} = \text{MSE}(\hat{S}, T) \tag{6}$$

$$\hat{S}_{ij} = \cos(\mathbf{e}_i, \mathbf{e}_j) \tag{7}$$

$$T_{ij} = \begin{cases} +1 & \text{if action}_i = \text{action}_j \\ -1 & \text{otherwise} \end{cases} \tag{8}$$

where $\mathbf{e}_i$ is the projected 3840-dim embedding for batch element i.

Critical Bug: Zero-Gradient Contrastive Target. The original implementation used $T_{ij} \in \{0, 1\}$ (not $\{-1, +1\}$). At initialization, all embeddings are approximately orthogonal: $\cos(\mathbf{e}_i, \mathbf{e}_j) \approx 0$. For different-action pairs with target $T_{ij} = 0$, the MSE gradient is $2(\hat{S}_{ij} - 0) = 2 \cdot 0 = 0$ — identically zero from the start. The contrastive loss provided no signal for the first many epochs.

Fix: use $T_{ij} \in \{-1, +1\}$. For different-action pairs: gradient $= 2(\hat{S}_{ij} - (-1)) = 2(0+1) = 2$, immediately active.

Result (Stage 2): Responsiveness ratio = 0.065. The model learned action presence (action vs. no-action) but not action identity (forward vs. backward were nearly identical).

4.3 Stage 3b: Bug Fixes + Fresh Training

Stage 3 applied all three bug fixes and retrained from scratch:

1. Xavier uniform init for W_{out} (gain=0.01)
2. Contrastive targets $\in \{-1, +1\}$
3. Gamma cast to float32, $\text{lr}_{\text{temporal}} = 10^{-4}$

Result: Projected cos-sim(fwd, bwd) = -0.344 (was ≈ 0). Responsiveness ratio: 0.463 (7$\times$ improvement from Stage 2).

4.4 Stage 3c: Continued Training (ep 51–100)

Starting from Stage 3b, we continued training for 50 more epochs with the same hyperparameters, tracking the best checkpoint. Table 3 shows the progression.

Table 3: Stage 3c checkpoint analysis.

Epoch	Proj. cos_sim (fwd/bwd)	Inj. res. cos_sim (fwd/bwd)	Verdict
50 (3b)	-0.344	-0.105	IMPROVING
60	-0.109	-0.560	IMPROVING
70	-0.456	-0.693	WORKING
80	-0.476	-0.614	WORKING
90	$+0.122$	$+0.508$	IMPROVING
100	-0.222	-0.277	IMPROVING

Cosine LR Decay Regression. The cosine learning rate schedule reduced lr from 5×10^{-5} to $\sim3\times10^{-5}$ by epoch 90. At this LR, the contrastive gradient (λ_c=0.3) could no longer compete with the diffusion loss (λ=1.0), causing embeddings to drift back toward the diffusion-optimal (non-discriminative) state. Epoch 80 remained the best checkpoint.

K/V Amplification Mechanism. An unexpected finding: the injection layer's K and V projections learned to amplify action differences through cross-attention, even while W_{out} remained at its diffusion-loss equilibrium (max weight $\approx$ 0.008). The injection residual cosine similarity improved 5$\times$ (from -0.105

to -0.693) without any growth in W_{out}. This suggests the model found a different discrimination pathway than anticipated.

Final result (ep80 best): Responsiveness ratio = 0.5725.

5 FIFO-Diffusion Inference

For streaming inference, we adopt FIFO-Diffusion [3]: a queue of Q latent frames at noise levels $\sigma_0 > \sigma_1 > \cdots > \sigma_{Q-1}$. At each step:

1. Denoise all frames jointly (one forward pass of the world model)
2. Shift queue: discard the cleanest frame, prepend a new noisy frame
3. The output stream is the sequence of emitted clean frames

This enables infinite-length video generation at ∼1.0 gen-fps (32 denoising steps, 256×256 resolution, A100 GPU), without reprocessing the full context.

6 Results

6.1 Responsiveness Metric

We measure action conditioning quality using the responsiveness ratio:

$$\rho = \frac{\Delta_{\text{action}}}{\Delta_{\text{action}} + \Delta_{\text{baseline}} + \epsilon} \tag{9}$$

where $\Delta_{\text{action}} = \mathbb{E}[\|\mathbf{v}_{\text{fwd}} - \mathbf{v}_{\text{bwd}}\|_1]$ (forward vs. backward video) and $\Delta_{\text{baseline}} = \mathbb{E}[\|\mathbf{v}_{\text{fwd}} - \mathbf{v}_{\text{none}}\|_1]$ (forward vs. no-action). $\rho > 0.5$ means action differences dominate over noise differences.

Table 4: Action responsiveness across training stages.

Stage	Δ_{action}	Δ_{baseline}	Ratio ρ
Stage 2	0.0009	0.0064	0.065
Stage 3b	0.0071	0.0082	0.463
Stage 3c (ep80)	0.0103	0.0077	0.5725

6.2 Training Loss Curves

Table 5 summarizes the loss progression across training stages.

Table 5: Training loss summary by stage.

Stage	Data	Epochs	Final Loss
Stage 1	Diverse video	10	0.704
Stage 2	GameFactory	30	0.537
Stage 3	GameFactory	50	0.507
Stage 3b	GameFactory	50	0.606
Stage 3c	GameFactory	100	0.580

Table 6: Projected action embedding similarities (ep80).

Action pair	Cosine similarity
forward / backward	−0.476
forward / run	−0.148
idle / jump	−0.140
forward / left	−0.092
left / right	+0.697

6.3 Action Embedding Analysis

After Stage 3c ep80, projected (3840-dim) cosine similarities between key action pairs:

Forward/backward are strongly anti-correlated (−0.476), as desired. Left/right remain similar (+0.697) — an open problem, likely due to limited left/right training examples in GameFactory.

7 Training Bugs and Fixes

We document three critical bugs that together caused 10+ epochs of wasted compute and a 7× suppression of the responsiveness ratio.

7.1 Bug 1: Zero-init Deadlock in ActionInjection

Symptom: Gate sigmoid remains at 0.499 after 30 epochs of training; to_out max weight < 0.001.

Root cause: $W_{\text{out}} = \mathbf{0}$ at init $\Rightarrow$ residual $= \mathbf{0} \Rightarrow \nabla_{\text{gate}} L = 0$.

Fix: Xavier uniform init with gain=0.01.

7.2 Bug 2: Contrastive Target in $\{0, 1\}$ Instead of $\{-1, +1\}$

Symptom: Projected cos-sim(fwd, bwd) ≈ 0 after 30 epochs; contrastive loss does not decrease.

Root cause: $T_{ij} = 0$ for different pairs; at init cos-sim ≈ 0; gradient $= 2(0 - 0) = 0$.

Fix: Change target $T_{ij} \in \{-1, +1\}$ so gradient is $2(0 - (-1)) = 2 \neq 0$ at init.

7.3 Bug 3: bfloat16 Gamma Freeze

Symptom: 0/10 temporal gammas change during training ("all unchanged" in analysis).

Root cause: $\gamma \approx 0.025$ in bfloat16; minimum representable step $\approx 1.2 \times 10^{-4}$; learning rate $5 \times 10^{-6} < 1.2 \times 10^{-4}$.

Fix: Cast gamma tensors to float32; use $\text{lr}_{\text{temporal}} \geq 10^{-4}$.

8 Discussion

8.1 What Makes This Hard

The core challenge is that the 6.15B frozen base model generates visually plausible frames regardless of action conditioning. The training signal for action discrimination must overcome the base model's strong spatial prior. At the current scale ($\Delta_{\text{action}} \approx 1\%$ pixel change), action conditioning is a small perturbation.

8.2 K/V Amplification as Alternative Mechanism

A surprising finding: the injection residual improved significantly ($-0.105 \rightarrow -0.693$) without any growth in W_{out}. The K/V projections of the cross-attention amplified action-specific signals through the attention mechanism, bypassing the need for large W_{out} values. This suggests that cross-attention architectures have multiple routes to learn conditioning.

8.3 Limitations

- Small absolute differences: $\Delta_{\text{action}} = 0.0103$ ($\sim$1% pixel change). For practical game use, we target $> 5\%$.
- Left/right confusion: cos-sim(left, right) = +0.697 suggests the model does not distinguish these well, likely due to limited training data coverage.
- Cosine LR decay: Embeddings regress in the final 20 epochs as contrastive gradient weakens; best checkpoint must be selected manually.
- Inference speed: $\sim$1 gen-fps is too slow for interactive simulation (target: 30 fps).

8.4 Future Work

1. Dual-forward contrastive loss: Run two forward passes per batch step (different actions), add loss directly on video output pixel differences. Provides gradient signal at the video level, not just embedding level.
2. Unfreeze base transformer: Allow Z-Image spatial layers to adapt at lr $\sim 10^{-6}$. Could dramatically increase absolute action magnitude.
3. ControlNet-style injection at every block: Current injection at 3 out of 30 layers is sparse. Adding injection at all 30 layers (via a trainable copy of the transformer, as in ControlNet) could provide denser action signal.
4. Cyclic LR for contrastive phases: Avoid cosine decay degrading embeddings by using cyclic LR or a constant-LR fine-tuning phase focused on contrastive loss.
5. Synthetic action data: Our generate_action_synthetic.py generates offline training data with guaranteed action-visual correspondence. Pre-training on this before fine-tuning on GameFactory may improve data efficiency.

9 Checkpoints and Code

All checkpoints are saved at:

- .../zimage_stage3c/world_model_s2_epoch80.pt
  Best checkpoint ($\rho = 0.5725$)
- .../zimage_stage3b/world_model_s2_final.pt
  Stage 3b baseline ($\rho = 0.463$)
- .../zimage_stage2_gamefactory/world_model_s2_final.pt
  Stage 2 ($\rho = 0.065$)

Code is organized as:

- models/zimage_world_model.py: ZImageWorldModel, TemporalAttention, ActionInjection
- training/action_finetune.py: ActionConditioningLoss with contrastive loss
- scripts/train_zimage_stage2.py: Main training script
- inference/fifo_pipeline.py: FIFO-Diffusion streaming inference
- scripts/analyze_stage3_checkpoint.py: Checkpoint analysis (embeddings, gates, gammas)
- scripts/measure_action_responsiveness.py: Responsiveness metric computation

- patches/apply_patches.sh: Re-apply diffusers/-transformers compatibility patches

10 Conclusion

We presented Z-Image World Model, an action-conditioned video generation system for game simulation. By attaching lightweight trainable modules (784M parameters) to a frozen 6.15B image diffusion transformer, we achieved a responsiveness ratio of 0.5725, demonstrating that player actions measurably affect generated video. Three critical training bugs were identified and fixed, yielding a 7× improvement in responsiveness. An unexpected K/V amplification mechanism was observed, where cross-attention key-value projections learned action discrimination without growing W_{out} weights. Future work focuses on increasing the absolute magnitude of action effects toward the $> 5\%$ pixel change target needed for practical game simulation.

References

[1] Jake Bruce et al. Genie: Generative interactive environments. arXiv preprint arXiv:2402.15391, 2024.

[2] Edward J. Hu et al. LoRA: Low-rank adaptation of large language models. arXiv preprint arXiv:2106.09685, 2021.

[3] Jihwan Kim et al. FIFO-Diffusion: Generating infinite videos from text without training. arXiv preprint arXiv:2405.11473, 2024.

[4] Dani Valevski et al. GameNGen: Diffusion models are real-time game engines. arXiv preprint arXiv:2408.14837, 2024.

[5] Jiwen Wang et al. GameFactory: Creating new games with generative interactive videos. arXiv preprint arXiv:2501.08325, 2024.

[6] Mengjiao Yang et al. UniSim: Learning interactive real-world simulators. arXiv preprint arXiv:2310.06114, 2023.

[7] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. Adding conditional control to text-to-image diffusion models. ICCV, 2023.

[8] Fangqi Zhu et al. IRASim: Learning interactive real-robot action simulators. arXiv preprint arXiv:2406.14540, 2024.

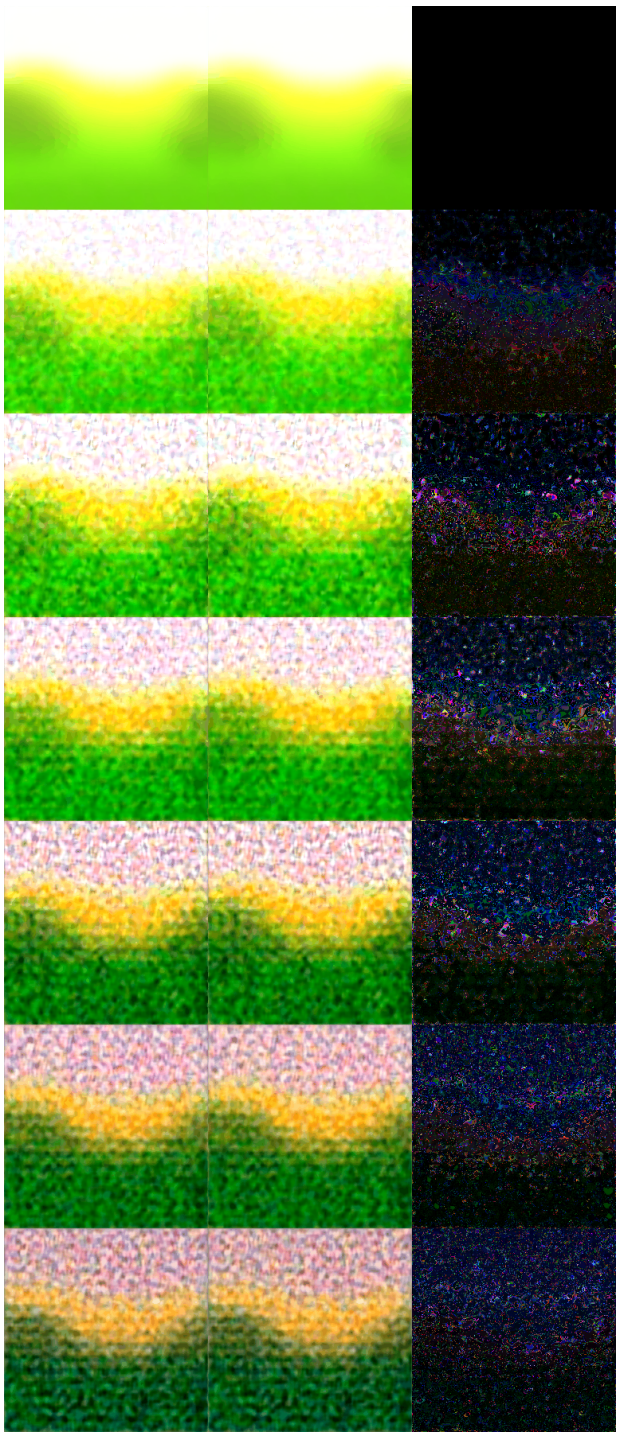


Figure 1: Comparison of generated video frames for FORWARD (left column), BACKWARD (middle column), and pixel-difference ×8 (right column) across 7 timesteps. Visible differences confirm that action conditioning affects generation.

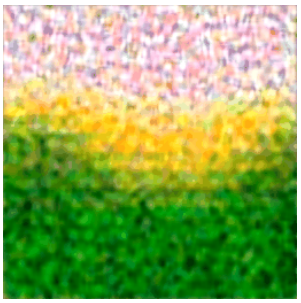
(a) FORWARD, frame 0

(b) BACKWARD, frame 0

(c) NO ACTION, frame 0

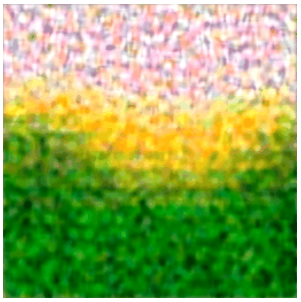
(d) JUMP, frame 0

Figure 2: Generated first frames for four actions using the Stage 3c ep80 model. Same random seed; differences arise purely from action conditioning.

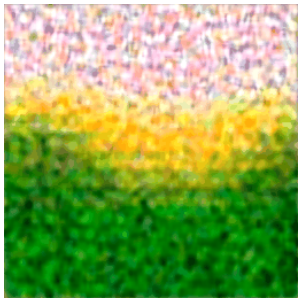


(a) FORWARD, frame 16

(b) BACKWARD, frame 16



(c) NO ACTION, frame 16



(d) JUMP, frame 16

Figure 3: Generated frames at timestep 16 (mid-sequence). Action-specific patterns become more pronounced over time.